

RELAZIONE CROSS

Diletta Di Domenico - 640804

10 NOVEMBRE 2025

Introduzione

La presente relazione illustra la struttura e l'organizzazione del progetto CROSS, un sistema client-server sviluppato in Java per offrire un servizio di orderBook dedicato al trading di criptovalute. Il lavoro è stato realizzato seguendo le specifiche contenute nel documento "ProgettoLab2425Versione1.3.pdf", con particolare attenzione alla separazione logica tra le componenti principali, pensata per garantire chiarezza, modularità e facilità di manutenzione.

L'obiettivo principale del progetto è la realizzazione di un sistema distribuito capace di gestire in modo concorrente le operazioni di più utenti, mantenendo al contempo coerenza dei dati, efficienza nelle comunicazioni e robustezza nel funzionamento.

Struttura

Il progetto è organizzato in una struttura gerarchica di cartelle compatibile con Maven.

Le componenti principali sono:

1. src, contiene codice sorgente suddiviso nei seguenti package:

- Json
 - *orderBook.json*, contiene tutti gli ordini di acquisto o di vendita con i relativi dati (utente, data, orderID...).
 - *storicoOrdini.json*, contiene tutti i dati relativi allo storico degli ordini.
 - *userMap.json*, contiene i dati relativi alle credenziali dell'utente registrati.
- Main/Java
 - Eseguibili
 - Client
 - *Printer.java*, gestisce la stampa dei messaggi asincrona, evitando di bloccare il thread principale.
 - *ReceiverClient.java*, riceve e interpreta i messaggi TCP provenienti dal server.
 - *UDP.java*, riceve notifiche UDP sugli ordini eseguiti.
 - Main
 - *MainClient.java*, gestisce la connessione TCP, la comunicazione con il server e la chiusura sicura dei thread.
 - *MainServer.java*, gestisce la connessione TCP dei client, i dati condivisi e la terminazione ordinata del server.
 - Server
 - *DailyParameters.java*, rappresenta i dati giornalieri di trading.
 - *SockMapValue.java*, memorizza le informazioni di rete associate ai client attivi.
 - *TimeoutHandler.java*, monitora l'attività dei client e gestisce le disconnessioni per inattività.
 - *Tupla.java*, gestisce l'autenticazione e lo stato di connessione degli utenti.
 - *Worker.java*, thread che gestisce la comunicazione con un singolo client, mantenendo sincronizzato lo stato del sistema e salvando i dati su file JSON.
 - Gson, racchiude le classi dedicate alla serializzazione e de-serializzazione dei dati in formato JSON
 - *GsonAskHistory.java*, comando per richiedere lo storico dei prezzi.
 - *GsonCredentials.java*, comando per gestire l'aggiornamento credenziali degli utenti.
 - *GsonLimitStopOrder.java*, comando per inserire un LimitOrder o uno StopOrder.
 - *GesonMarketOrder.java*, comando per inserire un MarketOrder.

- *GsonTrade.java*, rappresenta un trade letto dallo storico degli ordini.
- *GsonUser.java*, rappresenta un utente con relativo nome e password.
- *GsonOrderBook.java*, contiene l'orderBook o la lista degli StopOrders.
- *GsonResponse.java*, rappresenta la risposta generica del server.
- *GsonResponseOrder.java*, rappresenta la risposta relativa alle operazioni sugli ordini.
- *GsonMessage.java*, rappresenta i messaggi JSON.
- *Values.java*, valori generici dei messaggi JSON.
- *OrderBook*, contiene le classi che implementano la logica dell'orderBook
 - *Bookvalue.java*, rappresenta i campi della askMap o della bidMap.
 - *OrderBook.java*, implementa la struttura del mercato, gestendo ordini di bid e ask, il matching tra essi e le notifiche al client.
 - *StopValue.java*, rappresenta i campi della coda degli StopOrders.
 - *Trade.java*, rappresenta il trade inviato al client tramite la notifica UDP.
 - *UserBook.java*, rappresentare i campi delle liste di utente che hanno piazzato un ordine.

2. Target

- *classes*, include i file (.class) generati dalla compilazione, mantenendo le stesse strutture delle cartelle del codice src.
- *cross-client.jar* e *cross-server.jar*, file eseguibili utilizzati per avviare il programma.

client.properties, file di configurazione del client, indicando porta TCP e nome-host del server.

server.properties, file di configurazione del server, indicando porta TCP e UDP di ascolto del server e nome-host del server.

filenames.txt, file contenente l'elenco completo dei file sorgente del progetto, organizzati per percorso e suddivisi nei rispettivi package.

pom.xml, file di configurazione del progetto per Maven, gestendo le dipendenze e automatizzando la compilazione, l'esecuzione e la creazione dei JAR eseguibili. Sono stati configurati due profili di build, rispettivamente per il client e per il server.

Meccanismi di Comunicazione

Il progetto implementa due meccanismi di comunicazione tra client e server:

- **UDP**, impiegato per notificare in modo asincrono gli utenti sull'esecuzione delle transazioni relative agli ordini da loro inseriti nell'OrderBook, garantendo messaggi rapidi senza la necessità di mantenere una connessione persistente.
- **TCP**, è il principale mezzo di comunicazione client-server del sistema, utilizzato per instaurare una connessione stabile tra le due parti, tramite la quale il client invia le richieste al server e riceve le relative risposte. La connessione viene aperta all'avvio del client e rimane attiva fino alla chiusura del programma o del server.

Gestione della sincronizzazione

Il progetto adotta un approccio strutturato alla gestione della concorrenza in ambiente multithreading, per garantire l'integrità dei dati e il corretto funzionamento, permettendo il coordinamento efficiente delle operazioni di lettura e scrittura tra più thread.

Nel progetto sono state utilizzate le seguenti strutture dati concorrenti:

- *ConcurrentSkipListMap*, per memorizzare e gestire gli ordini di acquisto e vendita all'interno dell'OrderBook, permette di mantenere i dati ordinati e di eseguire inserimenti, ricerche e rimozioni in modo efficiente.
- *ConcurrentHashMap*, per gestire gli utenti registrati e le relative informazioni. Consente a più thread di leggere e modificare i dati senza blocchi globali.
- *ConcurrentLinkedQueue*, per gestire delle code di lavoro, come gli StopOrder e la lista dei worker attivi sul server, consentendo l'inserimento e la rimozione di elementi in parallelo, senza compromettere la consistenza dei dati.

OrderBook

L'OrderBook rappresenta la struttura centrale del sistema di mercato, responsabile della gestione e dell'organizzazione di tutti gli ordini di acquisto e di vendita inseriti dagli utenti.

Struttura

L'implementazione dell'OrderBook è stata effettuata in modo concorrenziale e thread-safe, consentendo agli utenti di operare simultaneamente senza generare conflitti. Per la gestione dei dati sono state utilizzate le strutture descritte in precedenza, tra cui:

- *askMap*, mappa chiave-valore in cui la chiave è un intero che rappresenta il prezzo, mentre il valore è un oggetto BookValue, memorizza gli ordini di vendita in modo crescente in base al prezzo.
- *bidMap*, mappa analoga alla askMap, ma dedicata agli ordini di acquisto, memorizzati in ordine decrescente di prezzo.
- *spread*, valore intero che rappresenta la differenza tra il miglior prezzo di acquisto e quello di vendita.
- *lastOrderID*, contatore utilizzato per assegnare in modo univoco un identificativo a ciascun ordine.
- *stopOrders*, coda i cui elementi sono oggetti StopValue descrivendo gli ordini stop inseriti dai client.

BookValue

La classe *BookValue* rappresenta il valore associato a un prezzo nelle mappe degli ordini (askMap e bidMap).

Permette di memorizzare:

- Size, quantità totale di ordini a quel prezzo.
- Total, valore totale corrispondente ($\text{total} = \text{size} * \text{prezzo}$).
- UserList, lista di utenti che hanno inserito ordini a quel prezzo.

Trade

La classe *Trade* rappresenta una transazione generata dall'esecuzione di un ordine all'interno dell'OrderBook.

Contiene le informazioni principali relative a ogni esecuzione e viene utilizzata per:

- Notificare gli utenti tramite UDP sull'esito dei loro ordini;
- Mantenere uno storico delle operazioni per eventuali log o analisi di mercato.

UserBook

La classe *UserBook* rappresenta un singolo ordine inserito da un utente all'interno dell'orderBook.

Ogni oggetto contiene le informazioni relative all'utente, alla quantità richiesta e all'ID dell'ordine, permettendo di tracciare in modo dettagliato gli ordini individuali.

Vengono memorizzati:

- *Size*, quantità dell'ordine inserito dall'utente
- *Username*, nome dell'utente che ha inserito l'ordine
- *Order_ID*, identificativo univoco dell'ordine
- *date*, data e ora di creazione dell'ordine

StopValue

La classe *StopValue* rappresenta StopOrder all'interno dell'OrderBook.

Gli ordini stop vengono eseguiti automaticamente quando il prezzo di mercato raggiunge o supera un determinato valore (*stopPrice*).

Lo scopo di questa classe è memorizzare tutte le informazioni necessarie per controllare, eseguire e notificare gli Stop Orders.

Vengono memorizzati:

- *Type*, tipo di ordine (bid, ask);
- *Size*, quantità dell'ordine stop;
- *Username*, nome dell'utente che ha inserito l'ordine;
- *Order_ID*, identificativo univoco dell'ordine;
- *stopPrice*, prezzo di attivazione dell'ordine stop.

Funzionamento

L'insieme delle classi descritte consente di gestire in modo completo e sincronizzato l'intero ciclo di vita di un ordine, dall'inserimento all'esecuzione.

Viene mantenuto un registro aggiornato degli ordini presenti, allineando le operazioni dei diversi utenti.

L'orderBook supporta tre categorie di ordini, ciascuna gestita da metodi dedicati che ne regolano l'inserimento e l'esecuzione:

- *LimitOrder*, rappresenta un ordine che viene eseguito solo a un prezzo specifico (massimo o minimo relativo al cui l'utente desidera comprare o vendere). Attraverso i metodi *tryBidOrder()* e *tryAskOrder()*, vengono elaborati gli ordini e cercando corrispondenze nella mappa opposta. Se l'ordine non viene completamente eseguito, la parte residua viene inserita nella mappa corrispondente tramite i metodi *loadBidOrder()* o *loadAskOrder()*.
- *MarketOrder*, è un ordine eseguito immediatamente al miglior prezzo disponibile sul mercato. Il metodo *tryMarketOrder()* si occupa della sua elaborazione, controllando la disponibilità all'interno dell'OrderBook, abbinando l'ordine con le offerte compatibili fino al suo completamento.
In caso di esecuzione parziale o fallimento, il sistema invia una notifica all'utente contenente un messaggio d'errore o di conferma.
- *StopValue*, è un ordine condizionato che si attiva solo quando il prezzo di mercato raggiunge o supera una soglia prefissata *stopPrice*. Gli ordini di questo tipo vengono aggiunti alla coda dedicata *stopOrders* e gestiti dal metodo *checkStopOrders()* invocato ogni volta che l'orderBook o la coda subiscono modifiche. Quando la condizione di prezzo viene soddisfatta, l'ordine viene automaticamente trasformato in un *MarketOrder* ed eseguito tramite *tryMarketOrder()*.

Quando due ordini opposti trovano una corrispondenza, l'orderBook genera una transazione rappresentata dall'oggetto *trade*, che contiene tutte le informazioni relative all'operazione, e viene utilizzato per aggiornare lo stato del sistema e per notificarlo agli utenti coinvolti.

Il funzionamento dell'OrderBook si fonda su un algoritmo di matching, incaricato di incrociare le proposte di acquisto con quelle di vendita secondo criteri di prezzo e priorità temporale.

Il metodo principale, *tryMatch()*, esamina gli ordini della parte opposta e procede seguendo queste fasi:

1. Scorre la lista degli ordini disponibili nell'altra sezione del book
2. Verifica che il contraente sia un utente diverso
3. Confronta quantità e gestisce i diversi casi:
 - Se l'ordine è superiore, l'ordine dell'utente viene eseguito completamente e la quantità residua resta nel book.
 - Se l'ordine è inferiore, viene completato e la parte restante dell'ordine principale rimane attiva.
 - In caso di uguaglianza, entrambi vengono chiusi.

Dopo ogni operazione, il sistema genera un oggetto *trade* con i dati della transazione e invia una notifica ai rispettivi utenti (UDP).

L'orderBook permette di svolgere altre funzionalità quali:

- Cancellazione, tramite il metodo *cancelOrder()* l'utente può rimuovere un proprio ordine non ancora eseguito.
- Aggiornamento dello stato, tramite il metodo *updateOrderBook()* vengono aggiornate in tempo reale le informazioni di mercato (Quantità totali, spread, prezzi migliori).
- Notifica all'utente, tramite il metodo *notifyUser* vengono inviate le notifiche UDP agli utenti i cui ordini sono stati evasi. Questo metodo individua l'indirizzo del client nella *socketMap* e se l'utente risulta connesso, procede inviando un pacchetto con i dettagli della transazione, dopo di che il messaggio viene serializzato in formato JSON tramite la libreria GSON e convertito in un array di byte, che viene trasmesso in rete in modo rapido e leggero.

Thread

Per garantire al sistema un'elevata reattività e un uso efficiente delle risorse, il progetto adotta un'architettura multi-thread sia sul lato client sia sul lato server.

Questo approccio permette di eseguire più operazioni in parallelo, garantendo un utilizzo efficiente delle risorse e una comunicazione di rete continua tra i due componenti. In pratica, ogni thread assume un compito specifico e opera in parallelo agli altri, consentendo di evitare blocchi o rallentamenti dovuti a operazioni di rete o a elaborazioni complesse.

Thread Client

Nel lato client, l'architettura multi-thread viene utilizzata per gestire in modo fluido la comunicazione con il server e per garantire un'interazione continua con l'utente, senza blocchi o ritardi.

L'obiettivo principale è quello di separare le responsabilità dell'applicazione, assegnando a ciascun thread un compito preciso, così da ottenere un flusso operativo parallelo e ben coordinato.

Componenti

I thread avviati all'interno del client sono:

- *MainClient*, è il thread principale e ha il compito di inizializzare il sistema, gestire l'interfaccia utente e inviare i comandi di connessione al server. Da questo thread vengono avviati i thread secondari, ognuno dedicato a una funzione specifica:
 - *ReceiverClient*, è il thread dedicato alla ricezione dei messaggi TCP provenienti dal server. Rimane costantemente in ascolto sul flusso di rete, interpretando i messaggi e aggiornando le variabili condivise.
 - *Printer*, è il thread dedicato alla stampa asincrona dei messaggi sulla console. Lavora in background mantenendo una coda di messaggi in attesa di visualizzazione, così da non interferire con le operazioni di rete o di input dell'utente.
 - *UDP*, è il thread dedicato alla ricezione dei messaggi UDP asincroni dal server. È progettato per gestire i pacchi di dati non persistenti, deserializzarli e notificare all'utente eventuali aggiornamenti.

Per consentire una corretta sincronizzazione tra i thread, vengono utilizzati diverse strutture di coordinamento:

- *BlockingQueue*, che permette di scambiare messaggi tra thread in modo ordinato e senza blocchi
- *Interrupt*, che permette di interrompere l'esecuzione dei thread in maniera controllata.
- *SharedData*, una classe che contiene le risorse condivise, protette tramite strutture thread-safe, per assicurare l'accesso escluso e sicuro ai dati.

Thread Server

Nel lato server, l'architettura multi-thread è progettata per gestire simultaneamente numerose connessioni client, ottimizzando al tempo stesso l'elaborazione delle richieste e la gestione delle risorse.

Componenti

I thread avviati all'interno del server sono:

- *MainServer*, è il thread principale del sistema e ha il compito di inizializzare il servizio, gestire le risorse e accettare nuove connessioni TCP in ingresso. Ogni volta che un nuovo client si connette, il Mainserver avvia un thread dedicato (*Worker*) all'interno di una *CachedThreadPool*, che consente di riutilizzare thread già terminati, riducendo l'overhead di creazione e distruzione.
 - *Worker*, è il thread responsabile della gestione completa di un singolo client e si occupa di:
 - Mantenere la comunicazione TCP bidirezionale con il client
 - Ricevere e interpretare i messaggi in formato JSON

- Eseguire le operazioni richieste (es. aggiornare l'orderBook)
- Aggiornare le mappe condivise di utenti e connessioni (userMap, orderBook ecc....)

Il sistema include anche il thread ausiliario:

- *TimeoutHandler*, che monitora l'attività dei client. Se una connessione rimane inattiva per un tempo superiore alla soglia impostata (10 minuti), la chiude automaticamente.

Per mantenere la coerenza dei dati condivisi tra i vari thread (userMap, socketMap, orderBook), il server impiega meccanismi di sincronizzazione basati su lock e strutture dati thread-safe.

In conclusione, sia sul lato client che su quello server, l'uso di thread indipendenti consente di suddividere le responsabilità operative, migliorando la reattività del sistema e riducendo i tempi di attesa nelle operazioni di rete.

Esecuzione del progetto

Per poter avviare il progetto è necessario posizionarsi nella directory cross ed eseguire i seguenti comandi:

```
java -jar target/cross-server.jar //Per eseguire il server
java -jar target/cross-client.jar //Per eseguire il client
```

UserMap - credenziali esistenti

All'interno del progetto è presente il file userMap.json contenente le credenziali degli utenti:

- username: admin, password: admin;
- username: diletta, password: psw;
- username: pin, password: o;